MIT 8103 Advanced Database Systems

IsaacMart Database Portfolio Summary

Idowu Isaac | Student ID: 30168852 | September Cohort 2026

Repository: https://github.com/LaProfessor/Idowu_Isaac_30168852_MIT8103

Case study and scope

IsaacMart is a simulated Nigerian e-commerce retailer. Customers place orders; warehouse staff manage products and stock; finance staff reconcile payments; managers analyse sales. PostgreSQL stores transactional data in Nigerian naira, while MongoDB stores flexible catalogue attributes. All records are synthetic. Delivery, taxes, refunds and customer authentication are outside this demonstration.

Portfolio 1: Database Design and Modelling

The six tables are Customers, Categories, Products, Orders, OrderItems and Payments. Primary and foreign keys identify and connect records. Unique constraints protect emails, SKUs and payment references; CHECK constraints reject negative stock, invalid quantities, prices and statuses. OrderItems resolves the order/product many-to-many relationship using the composite key (order_id, product_id).

Separating customer, product and category details removes partial and transitive dependencies, supporting 3NF under the stated rules. OrderItems stores historical unit_price so later product-price changes do not alter past sales. Totals are derived; cross-row payment rules require reconciliation. My Codespace checks confirmed 10,000 customers, 1,000 products, 200,000 orders, 200,000 lines and 180,000 payments. Integrity checks rejected negative stock, missing customers and duplicate emails. The folder includes schema, seed SQL, an ER diagram and normalisation notes.

Portfolio 2: Query Processing and Optimisation

The customer-history query retrieves the newest ten orders for one customer. A composite B-tree on (customer_id, ordered_at DESC) changed the plan from a parallel sequential scan and sort to an index scan. In my single Codespace comparison, execution time fell from 13.326 ms to 0.131 ms, approximately 102 times faster. Top-level execution buffers changed from 1,311 shared hits to 10 hits and 3 reads.

These are single-run results, not production predictions. Cache state and load affect timings; indexes cost storage and write maintenance. Separate assistant-run five-measurement medians were 12.802 ms before and 0.093 ms after indexing. I also ran category sales, monthly sales, customers-without-orders and reconciliation queries; reconciliation returned zero mismatches. Student-prefixed files retain my results and plans.

Portfolio 3: Transactions and Concurrency

COMMIT preserved a two-unit stock decrement from 100 to 98. A later five-unit decrement temporarily produced 93; ROLLBACK restored 98. In my two-session locking test, session A held a product-row lock while session B reached its two-second lock timeout. Rolling back A restored stock to 98. The initial assistant-run test used 500 ms. A timeout limits waiting; it does not establish deadlock prevention.

In REPEATABLE READ, session A read product 2 stock as 100. Session B committed 101, but A still read 100 from its snapshot. After A committed, its next query returned 101. I then restored stock to 100. READ COMMITTED instead obtains a new statement snapshot.

The corrected checkout transaction reserves stock, creates an order and line, and derives a pending payment from line totals. It aborts if the guarded stock update affects no row. My successful-path test produced matching order/payment values of NGN 1,050; rollback restored stock to 98 and left zero demo orders. Insufficient-stock rejection has not been separately tested. Remote payment settlement needs an idempotent outbox/saga workflow and reconciliation.

Portfolio 4: NoSQL and Advanced Data Models

MongoDB product documents embed category-specific attributes: battery hours for headphones, size/material for clothing and wattage for lamps. This accommodates varied descriptions without many nullable SQL columns. My run inserted three documents, retrieved the battery-hours match and updated an attribute. Duplicate-SKU and schema-validation tests rejected invalid inserts with codes 11000 and 121.

PostgreSQL owns prices, stock, orders and payments; MongoDB owns descriptive attributes. Cross-store synchronisation is proposed, not implemented. There are no cross-store foreign-key or atomic transaction guarantees.

Portfolio 5: Distributed and Cloud Database Exercise

Using the permitted discussion option, I consider a managed PostgreSQL primary, cross-zone standby and asynchronous reporting replicas, alongside a MongoDB replica set. Synchronous replication can reduce acknowledged-write loss but adds latency and may reduce write availability. Asynchronous replicas may lag; checkout must use authoritative stock. Fenced failover prevents conflicting writers. Replication does not replace backups and restoration drills.

Customer-based sharding could co-locate orders but complicates shared inventory and cross-shard checkout. Date partitioning supports retention without necessarily distributing data across servers. This dataset does not establish a need for sharding. No cloud deployment, replica-set experiment, sharding implementation or failover measurement was performed.

Evidence and repository

The private repository contains scripts, READMEs, the ER diagram, plans and execution outputs. Original assistant-run evidence is separate from Student-prefixed Codespace evidence. My initial verification was pushed in commit bf95174; corrected checkout evidence was added afterward. The complete first-session isolation sequence was shared during guided review; its saved final-state file records only the final value. Assessor access must be confirmed. Commit dates reflect actual work; no semester-long development history is claimed.

Individual technical reflection

I understood why sale-time prices must remain separate from current product prices. The index comparison showed that the same query can require very different amounts of work, although one timing comparison does not describe every workload. Commit, rollback, locking and repeatable read helped me distinguish temporary changes, committed changes and transaction snapshots.

Saving evidence was challenging: a relative output path failed inside psql because repository folders were outside the database container. With guidance, I saved output from the Codespace shell using tee. Coordinating two terminals also required keeping the first transaction open. I would improve the exercise by repeating performance measurements, testing more filters and verifying insufficient-stock rejection.

AI use declaration

I adopted IsaacMart as suggested by OpenAI ChatGPT/Codex. AI assisted with initial schemas, scripts, synthetic data, diagrams, explanations, analysis and documentation, and performed initial tests in its own environment. I subsequently executed setup, SQL queries, indexing, commit/rollback, locking, repeatable-read, MongoDB, integrity and corrected checkout checks in my Codespace with step-by-step AI guidance. I shared outputs for review and saved student evidence. AI also helped edit my reflection and correct technical wording. I remain responsible for reviewing, understanding and explaining the submission.

References

1. Miva University. MIT 8103 Advanced Database Systems: CA September Cohort 2026. Supplied assessment brief.

2. PostgreSQL Global Development Group. PostgreSQL 17 Documentation: Constraints, EXPLAIN, Multicolumn Indexes, Transaction Isolation, Explicit Locking and Warm Standby. https://www.postgresql.org/docs/17/

3. MongoDB. MongoDB Manual: Data Modeling, Schema Validation and Replication. https://www.mongodb.com/docs/manual/

Full topic-specific URLs appear in Idowu_Isaac_30168852_References.txt.